



EA551

Gerenciamento de processos em um sistema multiprogramado.

Ana Carolina Vieira Araújo 248734

Índice

1 Ajuste da atividade 3 e conversão de dados para .emat **2**

2 Visualização de imagens .emat com multiprogramação **3**

3 Multiplicação de matrizes .emat **10**

3.1 Gerenciamento da Memória e Leitura e Escrita 10

3.2 Lógica da Multiplicação e Execução: 11

3.3 Resultados Obtidos: 14

1 Ajuste da atividade 3 e conversão de dados para .emat

Para esta atividade, foi realizada a alteração do mecanismo de leitura e escrita no ficheiro ea551-mat.c, substituindo as funções de biblioteca padrão fread/fwrite por chamadas de sistema diretas read/write.

Posteriormente, foi desenvolvido um script em Bash para automatizar a conversão de todos os ficheiros de texto localizados na pasta mnist_txt para o formato binário proprietário .emat, preservando a estrutura de subpastas e gravando os resultados na pasta mnist_emat.

```bash
#!/bin/bash

# Define as pastas de origem (.txt) e destino (.emat)
ORIGEM="mnist_txt"
DESTINO="mnist_emat"

# Verifica se o executavel do programa existe na pasta atual
if [ ! -f ./ea551-mat ]; then
    echo "Executavel ./ea551-mat nao encontrado"
    exit 1
fi

echo "A iniciar a conversao de $ORIGEM para$DESTINO..."

# Busca todos os arquivos .txt e processa um por um
find "$ORIGEM" -type f -name "*.txt" | while read -r txt_file; do
    # Troca o caminho de 'mnist_txt' para 'mnist_emat'
    emat_file="${txt_file/$ORIGEM/$DESTINO}"
    # Troca a extensao de .txt para .emat
    emat_file="${emat_file%.txt}.emat"

    # Descobre o caminho da subpasta e cria a estrutura no destino
    dir_destino=$(dirname "$emat_file")
    mkdir -p "$dir_destino"

    # Executa a conversao redirecionando a entrada (.txt) e a saida (.emat)
    ./ea551-mat < "$txt_file" > "$emat_file"
done

echo "Conversao concluida. Ficheiros em '$DESTINO'."
```

Listing 1: Script de conversão para o formato .emat

Análise dos Ficheiros Gerados

```
00000000  45 4d 41 54 00 1c 00 00  00 1c 00 00 00 00 00 00  |EMAT............|
00000010  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000080  00 00 00 00 00 00 00 00  00 00 00 00 33 9f fd 9f  |............3...|
00000090  32 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |2...............|
```

```
00000000  45 4d 41 54 00 1c 00 00  00 1c 00 00 00 00 00 00  |EMAT............|
00000010  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
000000d0  00 00 00 00 00 00 00 00  00 00 00 00 00 37 94 d2  |.............7..|
000000e0  fd fd 71 57 94 37 00 00  00 00 00 00 00 00 00 00  |..qW.7..........|
```

Analisando os ficheiros gerados, é possível validar a correta estrutura do formato binário através dos seus 13 bytes iniciais de cabeçalho. Os primeiros 4 bytes exibem a sequência hexadecimal 45 4d 41 54, seguidos do quinto byte 00, que especifica o tipo de dado EA551_DTYPE_UINT8, e das sequências 1c 00 00 00 nos campos de linhas e colunas, que representam uma matriz de dimensões 28x28 em decimal.

```
ana-carolina@Carabina ~/Lab-Software-de-Sistemas (main)> cat mnist_txt/train/0/00001.txt
[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 51, 159, 253, 159, 50, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 48, 238, 252, 252, 252, 237, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 54, 227, 253, 252, 239, 233, 252, 57, 6, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 10, 60, 224, 252, 253, 252, 202, 84, 252, 253, 122, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 163, 252, 252, 252, 253, 252, 252, 96, 189, 253, 167, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 51, 238, 253, 253, 190, 114, 253, 228, 47, 79, 255, 168, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 48, 238, 252, 252, 179, 12, 75, 121, 21, 0, 0, 253, 243, 50, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 38, 165, 253, 233, 208, 84, 0, 0, 0, 0, 0, 0, 253, 252, 165, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 7, 178, 252, 240, 71, 19, 28, 0, 0, 0, 0, 0, 0, 253, 252, 195, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 57, 252, 252, 63, 0, 0, 0, 0, 0, 0, 0, 0, 0, 253, 252, 195, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 198, 253, 190, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 255, 253, 196, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 76, 246, 252, 112, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 253, 252, 148, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 85, 252, 230, 25, 0, 0, 0, 0, 0, 0, 0, 0, 7, 135, 253, 186, 12, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 85, 252, 223, 0, 0, 0, 0, 0, 0, 0, 0, 7, 131, 252, 225, 71, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 85, 252, 145, 0, 0, 0, 0, 0, 0, 0, 48, 165, 252, 173, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 86, 253, 225, 0, 0, 0, 0, 0, 0, 114, 238, 253, 162, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 85, 252, 249, 146, 48, 29, 85, 178, 225, 253, 223, 167, 56, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 85, 252, 252, 252, 229, 215, 252, 252, 252, 196, 130, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 28, 199, 252, 252, 253, 252, 252, 233, 145, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 25, 128, 252, 253, 252, 141, 37, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]
```

Figura 1: Matriz 28x28 correspondente.

2 Visualização de imagens .emat com multiprogramação

Foi feito o código que realiza a transformação das matrizes que foram transformadas de txt para emat para a representação em preto e branco.

A seguir uma explicação mais detalhada de partes importantes do funcionamento do visualizador.c

```
typedef struct __attribute__((packed)) {
    char magic[4];
    uint8_t dtype;
    uint32_t rows;
    uint32_t cols;
} EmatHeader;

typedef struct {
    uint32_t rows;
    uint32_t cols;
    uint8_t pixels[28 * 28];
} SharedData;

int shm_id = -1;
SharedData *shm_ptr = NULL;
sem_t *sem_pronto = NULL;
pid_t child_pid = -1;
```

Primeiro definimos a estrutura inicial para a leitura dos 13 bytes iniciais do aruivo .emat que vai ser recebido. O funcionamento dos atributos continua seguindo o mesmo explicado no roteiro anterior. (Assinatura, tipo e quantidade de linhas e colunas).

Após, é criada uma estrutura para a memória compartilhada entre os processos. Uma parte para a quantidade de linhas e colunas no arquivo, e um vetor que terá o valor dos pixels da imagem que será feita pelo processo pai.

Por último, a criação das variáveis globais de controle:

int shm_id guarda o id numérico único do segmento de memória. sharedData *shm_ptr é um ponteiro que armazena o endereço de memória virtual da memória compartilhada. sem_t *sem_pronto guarda o ponteiro para o semáforo criado em sem_open().

pid_t child_pid armazena o PID do processo filho retornado pelo fork().

```

void cleanup(void) {
    if (shm_ptr && shm_ptr != (void *)-1) {
        shmdt(shm_ptr);
    }
    if (shm_id != -1) {
        shmctl(shm_id, IPC_RMID, NULL);
    }
    if (sem_pronto && sem_pronto != SEM_FAILED) {
        sem_close(sem_pronto);
        sem_unlink(SEM_NAME);
    }
}

void sigint_handler(int sig) {
```

```
    (void)sig;
    if (child_pid > 0) {
        kill(child_pid, SIGTERM);
        waitpid(child_pid, NULL, 0);
    }
    cleanup();
    printf("\n[PAI] Encerramento por Ctrl+C concluido.\n");
    exit(0);
}
```

A primeira função é responsável por desalocar os recursos da memória compartilhada e semáforo alocados.

Verifica se a memória compartilhada foi anexada so endereço e se sim vai desanexa-la do processo atual. Ou seja, o processo vai perder o acesso ao ponteiro da memória. Depois da desanexação irá ter a destruição da memória da RAM.

Faz o mesmo processo de vericação e remoção para o semáforo, fechando o descritor do semáforo no processo e o removendo.

Para o sigint_handler, é enviado o sinal de término para o processo filho e bloqueia o pai até que o filho encerre completamente. O pai vai recolher o código de saída do filho para evitar acontecer um processo zumbi.

```

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Uso: %s <caminho_do_arquivo.emat>\n", argv[0]);
        return 1;
    }

    signal(SIGINT, sigint_handler);

    sem_unlink(SEM_NAME);
    sem_pronto = sem_open(SEM_NAME, O_CREAT | O_EXCL, 0666, 0);
    if (sem_pronto == SEM_FAILED) {
        perror("Erro ao criar semáforo sem_open");
        return 1;
    }

    shm_id = shmget(IPC_PRIVATE, sizeof(SharedData), IPC_CREAT | 0666);
    if (shm_id < 0) {
        perror("Erro ao criar memória compartilhada shmget");
        cleanup();
        return 1;
    }

    shm_ptr = (SharedData *)shmat(shm_id, NULL, 0);
    if (shm_ptr == (void *)-1) {
        perror("Erro ao anexar memória compartilhada shmat");
        cleanup();
```

```
        return 1;
    }

    child_pid = fork();

    if (child_pid < 0) {
        perror("Erro no fork");
        cleanup();
        return 1;
    }
```

Na main, caso haja um arquvio para ser processado, ocorre a inicialização das funções de tratamento e a criação do semáforo, com permissão de leitura e escrita.

O semáforo é criado com a flag 0 de valor inicial, para evitar condição de corrida caso o filho ser sem_wait() antes de o pai termianr de carregar os dados.**

Depois é alocado um bloco de memória do tamanho da estrutura SharedData com permissão de leitura e escrita. O id da memória alocada pelo shmget é passada para o shm_id e o bloco é passado para um ponteiro que irá apontar para essa memória.

Por fim, é feito o fork para a duplicação do processo. O processo filho terá a mesma tabela de páginas de memória e o ponteiro. No processo pai é retornoado o PID do filho criado.

```

    if (child_pid == 0) {

        sem_wait(sem_pronto);

        uint32_t orig_w = shm_ptr->cols;
        uint32_t orig_h = shm_ptr->rows;

        uint32_t win_w = orig_w * ESCALA;
        uint32_t win_h = orig_h * ESCALA;

        uint32_t *buf = calloc(win_w * win_h, sizeof(uint32_t));

        for (uint32_t y = 0; y < orig_h; y++) {
            for (uint32_t x = 0; x < orig_w; x++) {
                uint8_t gray = shm_ptr->pixels[y * orig_w + x];
                uint32_t pixel_argb = ((uint32_t)gray << 16) | ((uint32_t)gray
    << 8) | (uint32_t)gray;

                for (uint32_t sy = 0; sy < ESCALA; sy++) {
                    for (uint32_t sx = 0; sx < ESCALA; sx++) {
                        uint32_t px = x * ESCALA + sx;
                        uint32_t py = y * ESCALA + sy;
                        buf[py * win_w + px] = pixel_argb;
                    }
                }
            }
```

```
        }

        struct fenster f = {
            .title = "Visualizador .emat - Processo Filho",
            .width = win_w,
            .height = win_h,
            .buf = buf
        };

        fenster_open(&f);

        while (fenster_loop(&f) == 0) {
        }

        fenster_close(&f);
        free(buf);
        shmdt(shm_ptr);
        sem_close(sem_pronto);

        printf("[FILHO] Janela fechada pelo usuário. Encerrando filho.\n");
        exit(0);
```

Para o processo filho, ele dá sem_wait e fica esperando até o pai ler o .emat preencher a memória e chamar sem_post.

Depois de conseguir o semáforo, com acesso a memória compartilhada, vai ler os campos de linhas e colunas e redimensionar a janela.

Usa as informações para saber o tamanho do espaço que deve ser alocado para alocar espaço para a biblioteca fenster renderizar a janela.

Após é feito o processo de conversão de cor para cada pixel.

Após, há a criação da estrutura da janela gráfica do fenster. Quando for fechado acontece a liberação da RAM do buffer de pixels, oa desanexação da memória compartilhada a referência do semáforo e finaliza o processo filho.

```

        int fd = open(argv[1], O_RDONLY);

        if (fd < 0) {
            perror("Erro ao abrir arquivo .emat");
            kill(child_pid, SIGTERM);
            waitpid(child_pid, NULL, 0);
            cleanup();
            return 1;
        }

        EmatHeader header;
        if (read_all(fd, &header, sizeof(EmatHeader)) != (ssize_t)sizeof(
    EmatHeader)) {
```

```
            fprintf(stderr, "Erro ao ler cabeçalho do arquivo .emat\n");
            close(fd);
            kill(child_pid, SIGTERM);
            waitpid(child_pid, NULL, 0);
            cleanup();
            return 1;
        }

        shm_ptr->rows = header.rows;
        shm_ptr->cols = header.cols;

        size_t total_pixels = header.rows * header.cols;
        if (read_all(fd, shm_ptr->pixels, total_pixels) != (ssize_t)
    total_pixels) {
            fprintf(stderr, "Erro ao ler pixels do arquivo .emat\n");
            close(fd);
            kill(child_pid, SIGTERM);
            waitpid(child_pid, NULL, 0);
            cleanup();
            return 1;
        }

        close(fd);

        sem_post(sem_pronto);

        waitpid(child_pid, NULL, 0);

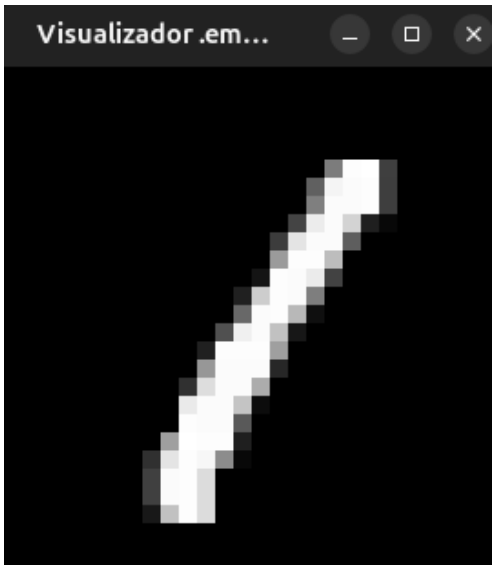
        cleanup();
        printf("[PAI] Processo filho finalizado. Pai encerrado.\n");
    }

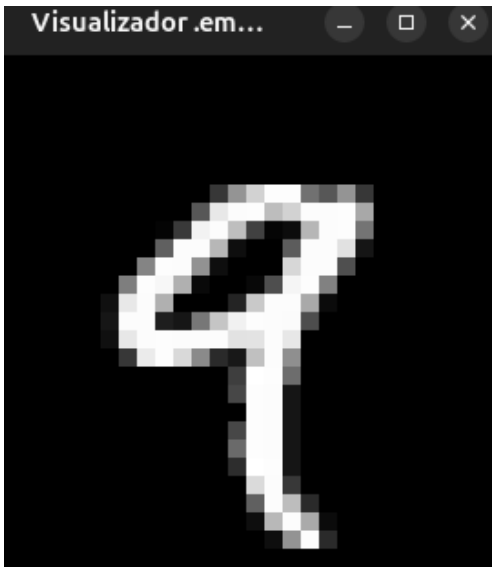
    return 0;
}
```

Para o processo pai, é feita a leitura do .emat, a estrutura do cabeçalho é instanciada e a função de leitura é chamada.

Posteriormente é feita a cópia dos valores para os seus campos correspondentes na memória compartilhada. A leitura dos bytes precisos é feita e o semáforo é liberado (sendo liberado para o processo filho), após o sinal de exit(0) do filho, a finalização do processo é feita.

Exemplos de saídas:


Visualizador .em...


Visualizador .em...


Visualizador .em...

3 Multiplicação de matrizes .emat

Para este exercício, foram criados os arquivos matmul.c e matmul.h, compilados e passado os arquivos do testes_4.3 - depois de passados pelo .emat - como argumentos.

As matrizes de índices i de A foram multiplicadas pelas matrizes B de mesmos índices e, em seguida, passadas por um sistema de comparação para verificar se o resultado condizia com os valores esperados nas matrizes C.

```c
#ifndef MATMUL_H
#define MATMUL_H

#include <stdint.h>
#include <stddef.h>

typedef struct __attribute__((packed)) {
    char magic[4];
    uint8_t dtype;
    uint32_t rows;
    uint32_t cols;
} EmatHeader;

typedef struct {
    uint32_t rows;
    uint32_t cols;
    double **data;
} Matrix;

Matrix *allocate_matrix(uint32_t rows, uint32_t cols);
void free_matrix(Matrix *mat);
Matrix *read_emat(const char *filename);
int write_emat(const char *filename, const Matrix *mat);
Matrix *multiply_matrices(const Matrix *A, const Matrix *B);
int compare_matrices(const Matrix *M1, const Matrix *M2, double tolerance);

#endif
```

Para o matmul.h foi criada a estrutura do cabeçalho do binário e a estrutura da representação da matriz. Além da declaração das funções que estarão no matmul.c.

3.1 Gerenciamento da Memória e Leitura e Escrita

```c
Matrix *allocate_matrix(uint32_t rows, uint32_t cols) {
    Matrix *mat = (Matrix *)malloc(sizeof(Matrix));
    if (!mat) {
        perror("Erro ao alocar estrutura Matrix");
        return NULL;
    }
    mat->rows = rows;
    mat->cols = cols;
```

```
9
10      mat->data = (double **)malloc(rows * sizeof(double *));
11      if (!mat->data) {
12          perror("Erro ao alocar ponteiros de linhas");
13          free(mat);
14          return NULL;
15      }
16
17      for (uint32_t i = 0; i < rows; i++) {
18          mat->data[i] = (double *)calloc(cols, sizeof(double));
19          if (!mat->data[i]) {
20              perror("Erro ao alocar colunas da matriz");
21              for (uint32_t j = 0; j < i; j++) free(mat->data[j]);
22              free(mat->data);
23              free(mat);
24              return NULL;
25          }
26      }
27      return mat;
28  }
29
30  void free_matrix(Matrix *mat) {
31      if (!mat) return;
32      if (mat->data) {
33          for (uint32_t i = 0; i < mat->rows; i++) {
34              if (mat->data[i]) free(mat->data[i]);
35          }
36          free(mat->data);
37      }
38      free(mat);
39  }
```

Com malloc, é reservado espaço na memória para a estrutura de controle matrix, além de espaço para o primeiro nível do ponteiro duplo, que vai guardar o vetor de ponteiros onde cada posição guardará o endereço do início de uma linha.

Após, é feita a a alocação da memória necessária para guardar os elementos da linha (quantidade de colunas).

3.2 Lógica da Multiplicação e Execução:

```
1  Matrix *multiply_matrices(const Matrix *A, const Matrix *B) {
2      if (A->cols != B->rows) {
3          fprintf(stderr, "Erro de dimensão: Colunas de A (%u) != Linhas de B (%
       u)\n", A->cols, B->rows);
4          return NULL;
5      }
6
7      Matrix *C = allocate_matrix(A->rows, B->cols);
8      if (!C) return NULL;
```

```
9
10      for (uint32_t i = 0; i < A->rows; i++) {
11          for (uint32_t k = 0; k < A->cols; k++) {
12              double r = A->data[i][k];
13              for (uint32_t j = 0; j < B->cols; j++) {
14                  C->data[i][j] += r * B->data[k][j];
15              }
16          }
17      }
18
19      return C;
20  }
```

Outro bloco importante é a parte da multiplicação das matrizes. Primeiramente, é verificado se a quantidade de colunas da primeira matriz é igual a quantidade de colunas da segunda, pois para multiplicar duas matrizes essa condição deve ser correspondida.

Se correspondido, será feita a alocação do espaço correspondente para a matriz c, que terá o tamanho das multiplicadas. É usado calloc para garantir que os espaços inciem preenchidos com 0.

Para o processor da multiplicação de fato, é feita a lógica de fixar um primeiro elemento da matriz A e, depois, multiplica-lo pelo elementos de mesma linha da matriz B, depois indo pro próximo e somando os resultados aos resultados anteriores continuamente. Os indices são controlados pelas variáveis. Com i controlando a linha de A e C, k controlando a coluna de A e a linha de B e j controlando a coluna de B e C.

```
1  int compare_matrices(const Matrix *M1, const Matrix *M2, double tolerance) {
2      if (M1->rows != M2->rows || M1->cols != M2->cols) {
3          printf("  [DIFERENÇA] Dimensões distintas: (%ux%u) vs (%ux%u)\n", M1->
       rows, M1->cols, M2->rows, M2->cols);
4          return 0;
5      }
6
7      for (uint32_t i = 0; i < M1->rows; i++) {
8          for (uint32_t j = 0; j < M1->cols; j++) {
9              if (fabs(M1->data[i][j] - M2->data[i][j]) > tolerance) {
10                 printf("  [DIFERENÇA] Na posição [%u][%u]: calculado=%f vs
       esperado=%f\n", i, j, M1->data[i][j], M2->data[i][j]);
11                 return 0;
12             }
13         }
14     }
15     return 1;
16 }
```

A função de comparação valida se a matriz calculada coincide com a do gabarito. Primeiro verifica se ambas possuem as mesmas dimensões, depois percorre elemento por elemento para verificar pelo valor absoluto se há uma diferença numérica entre elas.

```c
int main(int argc, char *argv[]) {
    if (argc < 3) {
        fprintf(stderr, "Uso: %s <A.emat> <B.emat> [C_esperado.emat]\n", argv[0]);
        return 1;
    }

    printf("\n=== Multiplicação de Matrizes .emat ===\n");
    printf("Lendo A: %s\n", argv[1]);
    Matrix *A = read_emat(argv[1]);
    if (!A) return 1;

    printf("Lendo B: %s\n", argv[2]);
    Matrix *B = read_emat(argv[2]);
    if (!B) {
        free_matrix(A);
        return 1;
    }

    printf("Calculando C = A (%ux%u) * B (%ux%u)...\n", A->rows, A->cols, B->rows, B->cols);
    Matrix *C = multiply_matrices(A, B);
    if (!C) {
        printf("[RESULTADO] Falha na multiplicação devido a erro de dimensão/alocação.\n");
        free_matrix(A);
        free_matrix(B);
        return 1;
    }

    printf("[SUCESSO] Multiplicação realizada! Matriz C resultante: %ux%u\n", C->rows, C->cols);

    if (argc >= 4) {
        printf("Comparando com o gabarito C esperado: %s\n", argv[3]);
        Matrix *C_esperado = read_emat(argv[3]);
        if (C_esperado) {
            if (compare_matrices(C, C_esperado, 1e-4)) {
                printf("[VERIFICAÇÃO OK] O resultado C é IDÊNTICO ao C_esperado!\n");
            } else {
                printf("[VERIFICAÇÃO ERRO] O resultado C difere do C_esperado.\n");
            }
            free_matrix(C_esperado);
        }
    }

    free_matrix(A);
    free_matrix(B);
    free_matrix(C);

```

```
47      return 0;
48  }
```

Na main temos o ponto de partida do programa, organizando a execução em passos simples: ela valida os nomes dos arquivos passados no terminal, lê as matrizes de entrada A e B e chama a função de multiplicação para calcular a matriz resultante C. Caso o usuário também forneça o caminho de uma matriz gabarito, ela faz a comparação automática dos valores e exibe na tela se o resultado foi correto ou se houve divergência, finalizando com a liberação de toda a memória alocada.

3.3 Resultados Obtidos:

```
=== Multiplicação de Matrizes .emat ===
Lendo A: testes_4.3/A_1.emat
Lendo B: testes_4.3/B_1.emat
Calculando C = A (32x24) * B (24x18)...
[SUCESSO] Multiplicação realizada! Matriz C resultante: 32x18
Comparando com o gabarito C esperado: testes_4.3/C_1.emat
[VERIFICAÇÃO OK] O resultado C é IDÊNTICO ao C_esperado!

=== Multiplicação de Matrizes .emat ===
Lendo A: testes_4.3/A_2.emat
Lendo B: testes_4.3/B_2.emat
Calculando C = A (10x8) * B (8x6)...
[SUCESSO] Multiplicação realizada! Matriz C resultante: 10x6
Comparando com o gabarito C esperado: testes_4.3/C_2.emat
[VERIFICAÇÃO OK] O resultado C é IDÊNTICO ao C_esperado!

=== Multiplicação de Matrizes .emat ===
Lendo A: testes_4.3/A_3.emat
Lendo B: testes_4.3/B_3.emat
Calculando C = A (2000x2000) * B (2000x2000)...
[SUCESSO] Multiplicação realizada! Matriz C resultante: 2000x2000
Comparando com o gabarito C esperado: testes_4.3/C_3.emat
[VERIFICAÇÃO OK] O resultado C é IDÊNTICO ao C_esperado!

=== Multiplicação de Matrizes .emat ===
Lendo A: testes_4.3/A_4.emat
Lendo B: testes_4.3/B_4.emat
Calculando C = A (2x3) * B (2x2)...
Erro de dimensão: Colunas de A (3) != Linhas de B (2)
[RESULTADO] Falha na multiplicação devido a erro de dimensão/alocação.
ana-carolina@Carabina:~/Lab-Software-de-Sistemas/LABORATÓRIO4/arquivos_atividade_extra_classe$ _
```